

MLCouncil — Codebase Algorithm Analysis (AS-IS / TO-BE)

Inspection Report

2026-05-20

Ispezione Codebase MLCouncil — Report Algoritmico AS-IS / TO-BE

Context

Parte 1 — Inventario algoritmi AS-IS

1.1 Stack a quattro layer

1.2 Mappa per file (riferimenti chiave)

Parte 2 — Teoria matematica per algoritmo

2.1 Feature engineering (Layer 2)

2.2 Modelli alpha (Layer 3)

2.3 Council aggregator (Layer 4)

2.4 Portfolio construction (CVXPY)

2.5 Esecuzione (Layer 4 — bottom)

2.6 Risk management

2.7 Performance metrics (backtest/report.py)

Parte 3 — Diagrammi

3.1 Diagramma di flusso end-to-end

3.2 Stati HMM

3.3 Diagramma di robustezza Conformal sizing

3.4 Risk waterfall

2.8 Monitoring & alerting (council/monitor.py)

2.9 Tabella iperparametri “nascosti” (non in CLAUDE.md)

Parte 4 — Analisi critica AS-IS (punti di debolezza)

Parte 5 — Proposte disruptive TO-BE

5.1 Modelli alpha di nuova generazione

5.2 Council aggregation non lineare

5.3 Portfolio construction evoluto

5.4 Esecuzione di nuova generazione

5.5 Risk management

5.6 Infrastruttura & MLOps

5.7 Roadmap proposta (priorità)

Verifica & utilizzo del report

Esecuzione: export in PDF (richiesto dall'utente)

Prossimi passi consigliati

Ispezione Codebase MLCouncil – Report Algoritmico AS-IS / TO-BE

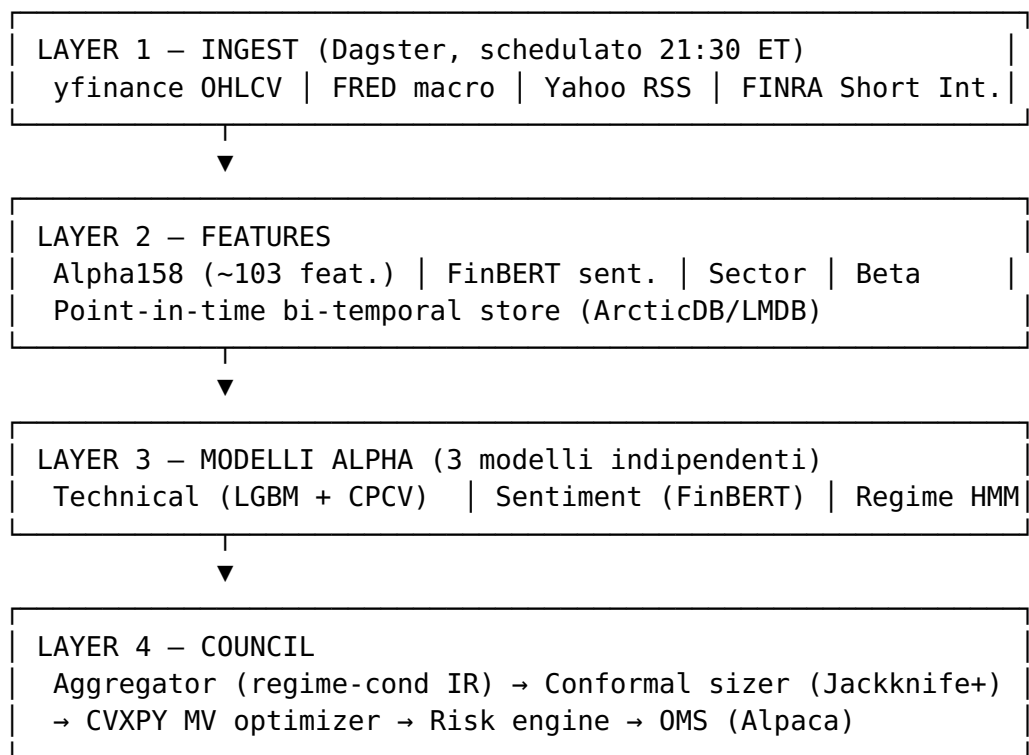
Context

L'utente ha richiesto un'ispezione completa della codebase MLCouncil (sistema di trading multi-modello con council ensemble) con: 1. **AS-IS**: mappa completa degli algoritmi presenti, con teoria matematica sottostante e diagrammi 2. **TO-BE**: proposte disruptive di evoluzione

Questo file è il **deliverable** del task: contiene direttamente il report (non un piano di codice da implementare). L'utente lo leggerà come output finale.

Parte 1 – Inventario algoritmi AS-IS

1.1 Stack a quattro layer



1.2 Mappa per file (riferimenti chiave)

Categoria	File	Cosa fa
Ingest OHLCV	data/ingest/ market_data.py:79-185	yfinance + halt detection + bi- temporal write
Ingest macro	data/ingest/macro.py: 135-180	VIX, yield spread DGS10-DGS2, S&P rolling returns
Ingest short int.	data/ingest/ short_interest.py: 104-126	FINRA si_ratio, days_to_cover, z- score
Feature Alpha158	data/features/ alpha158.py:69-396	~103 indicatori tecnici look-ahead safe
Target eng.	data/features/ target.py:53-158	forward returns risk-adjusted + winsorization 1/99
Sector exposure	data/features/ sector_exposure.py: 79-145	mapping settoriale + beta rolling 60d
Feature store	data/store/ arctic_store.py: 79-193	bi-temporal (valid_time + transaction_time)
Orchestrazione	data/pipeline.py: 395-1482	4 layer Dagster, retry policy, asset checks
Modello tecnico	models/technical.py	LightGBM + CPCV + SHAP/MLflow
Modello sentiment	models/sentiment.py	ProsusAI/finbert con cache SQLite
Modello regime	models/regime.py	HMM gaussiano 3- stati
Aggregator	council/ aggregator.py:200+	pesi regime- conditional + adaptive IR + ortogonalità
Conformal sizer	council/conformal.py	MAPIE Jackknife+ per scaling posizione
Portfolio	council/portfolio.py: 106-525	CVXPY MV con vincoli completi
Risk engine	council/ risk_engine.py:66-127	VaR/CVaR + breach detection
Risk rules	council/ risk_rules.py:51-190	

Categoria	File	Cosa fa
		stop-loss, trailing, time-based, DD circuit
Transaction costs	council/ transaction_costs.py: 32-117	Almgren-Chriss square-root model
Slicer	execution/slicer.py: 74-223	TWAP/VWAP adattivo, ADV-trigger
OMS	execution/oms.py: 177-393	lifecycle, partial fills, exec quality
Alpaca adapter	execution/ alpaca_adapter.py: 39-688	retry exponential, ADV check
Backtest	backtest/runner.py: 322-966	NautilusTrader, fill model next-open
Report metriche	backtest/report.py: 129-358	Sharpe, Calmar, IC, attribution

Parte 2 — Teoria matematica per algoritmo

2.1 Feature engineering (Layer 2)

Parkinson Volatility (alpha158.py:172-175)

Stimatore di volatilità basato su high/low (più efficiente del close-to-close): $\sigma_{Park}^2 = \frac{1}{4n \ln 2} \sum_{t=1}^n [\ln(H_t/L_t)]^2$ $\sigma_{Park}^2 = \frac{1}{4n \ln 2} \sum_{t=1}^n \left[\ln \left(\frac{H_t}{L_t} \right) \right]^2$ **Perché:** efficienza 5x rispetto a σ classico, robusto a gap overnight.

Cross-sectional rank stabile (alpha158.py:321-337)

Con denominatore fisso n_{max} (non n_{daily}) per evitare bias in giorni con universo sparso: $rank_{pct} = \frac{r_i - 1}{n_{max} - 1}$

Returns rolling stabili (macro.py:163-172)

Composizione log-stabile: $R_N = \exp(\sum_{t=1}^N \ln(1+r_t)) - 1$ $R_N = \exp(\sum_{t=1}^N \ln(1+r_t)) - 1$

VWAP deviation (alpha158.py:119-133)

$$\text{VWAP}_n = \frac{\sum_{i=1}^n c_i v_i}{\sum_{i=1}^n v_i}, \quad \text{dev}_t = |c_t - \text{VWAP}_n|$$
$$\text{dev}_t = \frac{c_t - \text{VWAP}_n}{|c_t|}$$

Indicatori momentum classici

- **RSI Wilder:** $\text{RSI} = 100 - \frac{100}{1 + \text{RS}}$, $\text{RS} = \frac{\text{gain}}{\text{loss}}$
- **MACD:** $\text{EMA}_{12}(c) - \text{EMA}_{26}(c)$, signal = EMA_9 del MACD
- **Bollinger:** $\mu_{20} \pm 2\sigma_{20}$, position = $\frac{(c-L)/(U-L)}{(c-L)/(U-L)}$
- **Williams %R:** $-100 \cdot \frac{(H_{14} - c)}{(H_{14} - L_{14}) - 100} \cdot \frac{(H_{14} - c)}{(H_{14} - L_{14})}$
- **Stochastic %K:** $100 \cdot \frac{(c - L_{14})}{(H_{14} - L_{14})} \cdot \frac{(c - L_{14})}{(H_{14} - L_{14})}$

Winsorization per-ticker (target.py:115-139)

Adattiva ai percentili 1/99 calcolati **per singolo ticker** (no cross-leakage): $\tilde{y}_t = \text{clip}(r_t + h\sigma_{21}, q_{0.01}, q_{0.99})$

2.2 Modelli alpha (Layer 3)

Technical — LightGBM + CPCV

Gradient Boosting: ensemble di alberi che minimizza la loss tramite discesa funzionale:

$$F_M(x) = \sum_{m=1}^M v_m \cdot h_m(x), \quad h_m = \arg\min_h \sum_i L(y_i, F_{m-1}(x_i) + h(x_i))$$
$$F_M(x) = \sum_{m=1}^M v_m \cdot h_m(x), \quad h_m = \arg\min_h \sum_i L(y_i, F_{m-1}(x_i) + h(x_i))$$

Iperparametri esatti (config/models.yaml:1-12): -
n_estimators=500, learning_rate=0.05, num_leaves=64 -
min_child_samples=20, subsample=0.8, colsample_bytree=0.7 -
Regularizzazione: $\lambda_{L1}=0.1$, $\lambda_{L2}=0.1$ - random_state=42, early stopping con **patience=50** su tail 15% del train (technical.py:244-261)

Combinatorial Purged Cross-Validation (López de Prado):
divide il training set in $N=6$ gruppi, prende $k=2$ come test set $\rightarrow \binom{6}{2} = 15$ combinazioni (technical.py: 78-99). Per ogni fold: - **Purging:** elimina osservazioni con valid_time overlapping con test - **Embargo:** rimuove ulteriori embargo_days=5 posizioni prima del test set dal train - Seleziona

modello con highest **OOF IC** (Spearman cross-section)
 (technical.py:269-271) - Fallback: se nessun fold valido → train su full dataset (technical.py:274-276)

SHAP (Shapley additive explanations): attribution per feature con valori di Shapley dalla teoria dei giochi cooperativi (sample 500 o 15% del dataset, technical.py:298-299): $\phi_i = \sum_{S \subseteq F \setminus \{i\}} |S|! (|F| - |S| - 1)! |F|! [f(S \cup \{i\}) - f(S)]$ $\phi_i = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)]$

Output: z-score cross-sezionale per data (technical.py:368-370): $\hat{s}_i = \frac{s_i - \mu_{date}}{\sigma_{date}}$ se $\sigma = 0$ $\hat{s}_i = \frac{s_i - \mu_{date}}{\sigma_{date}}$ se $\sigma = 0$

Sentiment — FinBERT

Architettura **Transformer encoder** pre-addestrata su corpus finanziario (ProsusAI/finbert, fallback yiyanghkust/finbert-tone).
 Output: probabilità su {positive, neutral, negative},
 max_length=512, batch=32, device GPU se disponibile (sentiment.py: 88-116).

Attention multi-head: $\text{Attn}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$
 $\text{Attn}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$

Conversione a scalare (sentiment.py:151): $\text{score}_h = P(\text{positive} | h) - P(\text{negative} | h) \in [-1, +1]$ $\text{score}_h = P(\text{positive} | h) - P(\text{negative} | h) \in [-1, +1]$

Aggregazione per ticker con recency decay (sentiment.py: 376-431): $\bar{s}_i = \frac{\sum_h \text{score}_h \cdot \gamma_{dh} \cdot w_{src}(h)}{\sum_h \gamma_{dh} \cdot w_{src}(h)}$ $\bar{s}_i = \frac{\sum_h \text{score}_h \cdot \gamma_{dh} \cdot w_{src}(h)}{\sum_h \gamma_{dh} \cdot w_{src}(h)}$ con $d_h = \max(0, \text{ref_date} - \text{pub_date}_h)$ $d_h = \max(0, \text{ref_date} - \text{pub_date}_h)$, $w_{src}(h) = \text{credibilità della fonte}$.

Calibrazione di γ : ottimizzazione bounded Brent (scipy.minimize_scalar) su $\gamma \in [0.30, 0.95]$ $\gamma \in [0.30, 0.95]$ per massimizzare $|\text{IC Spearman}|$ tra sentiment aggregato e forward return (sentiment.py:206-314).
 Threshold: se < 5 ticker con IC valido → ritorna 0.0.

Aggregation cross-sectional z-score (solo ticker con news, poi zero-fill, sentiment.py:350-364): $s_i = \bar{s}_i - \mu + \epsilon$ $s_i = \frac{\bar{s}_i - \mu}{\sigma + \epsilon}$

Cache SQLite con hash SHA-256 della headline evita re-scoring (news_processor.py:165-222).

Regime — Hidden Markov Model gaussiano

3 stati latenti {bull,bear,transition}\{\text{bull}, \text{bear}, \text{transition}\} con emissioni gaussiane $x_t \sim \mathcal{N}(\mu_k, \Sigma_k)$ $\mathbf{x}_t \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$. Covariance type=full (3×3 piene), n_iter=100, random_state=42 (config/models.yaml:19-23). Backend: hmmlearn.GaussianHMM, fallback sklearn.mixture.GaussianMixture (regime.py:40-86).

Feature input ordinate per preferenza (regime.py:94-109): [sp500_ret_21d, vix_level, yield_spread] → fallback a varianti più semplici. Preprocessing: ffill().bfill().fillna(0) + StandardScaler() (regime.py:195-198).

Forward-backward per inference: $\alpha_t(k) = P(x_{1:t}, z_t = k)$, $\alpha_t(k) = \mathcal{N}(x_t | \mu_k, \Sigma_k) \sum_j \alpha_{t-1}(j) A_{jk}$ $\alpha_t(k) = P(\mathbf{x}_{1:t}, z_t = k)$, $\alpha_t(k) = \mathcal{N}(x_t | \mu_k, \Sigma_k) \sum_j \alpha_{t-1}(j) A_{jk}$

Baum-Welch (EM) per training: massimizza $\log P(X|\theta)$ $\log P(X|\theta)$ alternando E-step (forward-backward) e M-step (update di A, μ_k, Σ_k $A, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k$).

State labelling deterministico (regime.py:137-183): ordina stati per μ_k μ_k del primo feature (equity return) → state con μ μ massimo = bull, minimo = bear, intermedi = transition. Evita ambiguità da numerazione arbitraria HMM tra retraining.

Training settimanale domenicale 23:00 ET (pipeline.py:952-971), checkpoint con hash SHA256 sidecar.

2.3 Council aggregator (Layer 4)

Regime-conditional weighting

Pesi base da config/regime_weights.yaml:
 $w_{\text{modelbase}} = w_{\text{regime}[\text{model}]} w_{\text{model}}^{\text{base}} = w_{\text{regime}[\text{model}]}$

Adaptive Information Ratio (EWM, halflife=20)

Rolling 100-day, applicato dopo min_history_days=30. NB: **non** è media semplice ma **EWM** con halflife=min(20, len(recent)//2) (aggregator.py:514) — peso $\sim 4\times$ su mese recente vs 60 giorni fa:
 $\text{IC}^{\text{mEWM}} = \text{EWM}_{hl=20}(\text{IC}_m)$, $\sigma^{\text{mEWM}} = \text{EWM}_{hl=20}(\text{std}(\text{IC}_m))$
 $\overline{\text{IC}}^{\text{EWM}}_m = \text{EWM}_{hl=20}(\text{IC}_m)$, $\sigma^{\text{EWM}}_m = \text{EWM}_{hl=20, \text{std}}(\text{IC}_m)$
 $\text{Sharpe}_m = \frac{\overline{\text{IC}}^{\text{EWM}}_m}{\sigma^{\text{EWM}}_m + \epsilon} \sqrt{252}$, $w_m^{\text{adj}} =$

$w_m^{\text{base}} \cdot \max(0.1, \text{Sharpe}_m)$ **Soft floor 0.1**
 (aggregator.py:437): un modello con Sharpe negativo non viene
 azzerato ma capped a $0.1 \cdot w_m^{\text{base}}$ per
 permettere recovery. Clipping finale $[0.05, 0.70]$ via
 projection sul simplex vincolato in $O(n)$ (aggregator.py:
 454-493).

Orthogonality constraint

Su correlazione rolling 60-day tra signal: if $|\rho_{ij}^{60d}| > 0.70 \Rightarrow w_{\min(i,j)} \leftarrow w_{\min(i,j)} \cdot 0.5$
 $\text{if } |\rho_{ij}^{60d}| > 0.70 \Rightarrow w_{\min(i,j)} \leftarrow w_{\min(i,j)} \cdot 0.5$

Conformal prediction (MAPIE Jackknife+)

Base estimator **Ridge($\alpha=1.0$)**,
 CrossConformalRegressor(method="plus"), 5-fold CV, random_state=42,
 coverage target 0.85 $\rightarrow \alpha=0.15$ (conformal.py:47-99).

Per ogni osservazione i , calibra il residuo leave-one-out: $R_i = |y_i - \hat{f}_{-i}(x_i)|$
 $R_i = |y_i - \hat{f}_{-i}(x_i)|$ Predizione con
 intervallo: $\hat{C}_n^\alpha(x) = [\hat{f}(x) - q_{1-\alpha}(\{R_i\}), \hat{f}(x) + q_{1-\alpha}(\{R_i\})]$
 $\hat{C}_n^\alpha(x) = [\hat{f}(x) - q_{1-\alpha}(\{R_i\}), \hat{f}(x) + q_{1-\alpha}(\{R_i\})]$
 Garanzia di copertura
 finita: $P(y \in \hat{C}_n^\alpha(x)) \geq 1 - 2\alpha n + 1$
 $P(y \in \hat{C}_n^\alpha(x)) \geq 1 - 2\alpha n + 1$ senza assunzioni distribuzionali.

Position multiplier — mapping esponenziale (conformal.py:
 139-188), non reciprocale:
 $\text{widthnorm} = u - \text{lmedian}(u - l)$, $m = \text{clip}(e^{1 - \text{widthnorm}}, 0.2, 2.0)$
 $\text{widthnorm} = \frac{u - l}{\text{median}(u - l)}$, $m = \text{clip}(e^{1 - \text{widthnorm}}, 0.2, 2.0)$
 Scelta motivata: il vecchio $1/\text{widthnorm}$
 saturava in una "dead zone" per intervalli stretti ($\text{widthnorm} \leq 0.5$).
 L'esponenziale dà decay smooth senza saturazione.

Filtro low-confidence (conformal.py:194-233): segnali con $\text{width} \geq \text{percentile}(\text{widths}, 90)$ vengono azzerati (drop del 10% peggiore).

2.4 Portfolio construction (CVXPY)

Problema convesso completo (portfolio.py:257-456):
 $\max_w (\alpha \odot m)^T w - \lambda \text{tc} \|w - w_{\text{curr}}\|_1$ s.t. $\mathbf{1}^T w = B(\text{budget fraction})$
 $0 \leq w \leq \text{umax}(\text{long-only} + \text{cap})$, $\|w - w_{\text{curr}}\|_1 \leq T \text{max}(\text{turnover})$
 $w^T \Sigma w \leq \sigma \text{daily}^2(\text{vol cap})$, $S^T w \leq c \text{sec}(\text{sector cap})$, $|\beta^T w| \leq \beta \text{max}(\text{beta neutrality})$
 $\begin{aligned} \max_w & \quad (\alpha \odot m)^T w - \lambda \text{tc} \|w - w_{\text{curr}}\|_1 \\ \text{s.t.} & \quad \mathbf{1}^T w = B \\ & \quad 0 \leq w \leq \text{umax} \\ & \quad \|w - w_{\text{curr}}\|_1 \leq T \text{max} \\ & \quad w^T \Sigma w \leq \sigma \text{daily}^2 \\ & \quad S^T w \leq c \text{sec} \\ & \quad |\beta^T w| \leq \beta \text{max} \end{aligned}$

$$\begin{aligned} \sigma_{\text{daily}}^2 &\quad \text{(vol cap)} \\ \leq c_{\text{sec}} &\quad \text{(sector cap)} \\ \beta_{\text{max}} &\quad \text{(beta neutrality)} \end{aligned}$$

Solver default + fallback **SCS** (Splitting Conic Solver).

Ledoit-Wolf shrinkage (pipeline.py:1545-1554)

$\hat{\Sigma}_{LW} = \delta \cdot F + (1 - \delta) \cdot S$ con $FF = \text{target strutturato (matrice di covarianza a singolo fattore o identità scalata)}$ e δ ottimo analitico che minimizza la perdita di Frobenius attesa.

Capped simplex projection (portfolio.py:218-254)

Post-processing dei pesi greedy via binary search sulla soglia τ : $w_i = \text{clip}(v_i - \tau, 0, u_{\text{max}})$, $\tau^* : \sum w_i = \bar{B}$ 100 iterazioni $\rightarrow$ precisione macchina.

2.5 Esecuzione (Layer 4 — bottom)

Almgren-Chriss square-root market impact (transaction_costs.py:32-72)

$\text{slippage}_{\text{bps}} = \sigma \cdot QV \cdot \kappa$ Implementato come lookup table per ticker $\times$ volume factor $(V_{\text{ref}}/V_{\text{realized}})^{0.3}$, bounds $[0.5, 2.0]$.

Cost stimation (transaction_costs.py:93-117)

$\text{cost} = \sum |w_t - w_{t-1}|^2 \cdot c_{\text{bps}} + s_{\text{bps}} \cdot PV$

VWAP execution profile (execution/slicer.py)

Volume profile a 7 bucket orari (mattina 20%, pre-close 30%). Adaptive slicing $n \in [4, 16]$ in base a urgency.

Almgren-Chriss completo (non implementato ma riferimento teorico)

Trade-off ottimo tra costo permanente e rischio temporale:

$$\min_k \mathbb{E}[X] + \gamma \cdot \text{Var}[X], X = \sum_k \tau \cdot \sigma^2 \cdot x_k^2 + \eta \cdot n_k^2$$

$$\mathbb{E}[X] + \gamma \cdot \text{Var}[X], \quad X = \sum_k \tau \cdot \sigma^2 \cdot x_k^2 + \eta \cdot n_k^2$$

2.6 Risk management

Value-at-Risk (risk_engine.py:66-89)

- **Historical:** $\text{VaR}_\alpha = -\text{quantile}_\alpha(r_{1:T})$ $\text{VaR}_\alpha = -\text{quantile}_\alpha(r_{1:T})$
- **Parametric:** $\text{VaR}_\alpha = -(\mu - z_\alpha \sigma)$ $\text{VaR}_\alpha = -(\mu - z_\alpha \sigma)$
- **Monte Carlo:** simulazione NN traiettorie da modello di ritorni

Expected Shortfall (CVaR)

$$\text{CVaR}_\alpha = -\mathbb{E}[r | r \leq -\text{VaR}_\alpha] \quad \text{CVaR}_\alpha = -\mathbb{E}[r \mid r \leq -\text{VaR}_\alpha]$$

Drawdown circuit breaker (risk_rules.py:170-190)

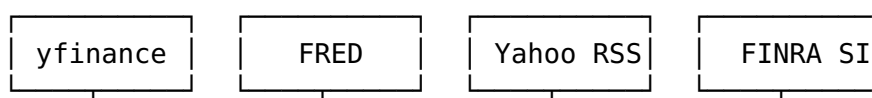
$$\text{cash_frac} = \{0.50\text{se} \\ \text{DD} > \text{DD}_{\text{max}} \text{cmin} + \text{DD} - \text{DD}_{\text{warn}} \text{DD}_{\text{max}} - \text{DD}_{\text{warn}} (0.5 - \text{cmin}) \text{se} \text{DD}_{\text{warn}} < \text{DD} \leq \text{DD}_{\text{max}}$$

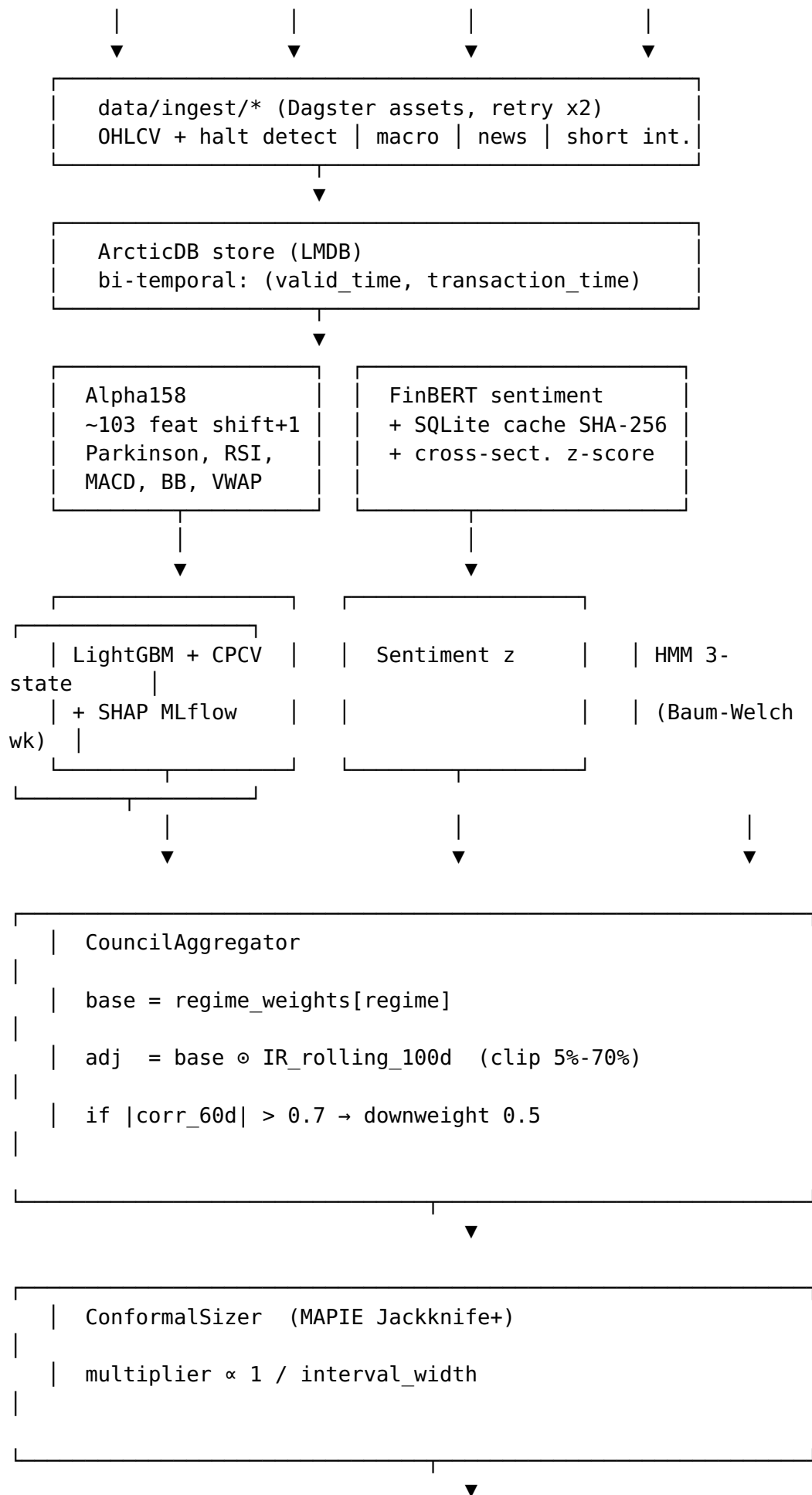
2.7 Performance metrics (backtest/report.py)

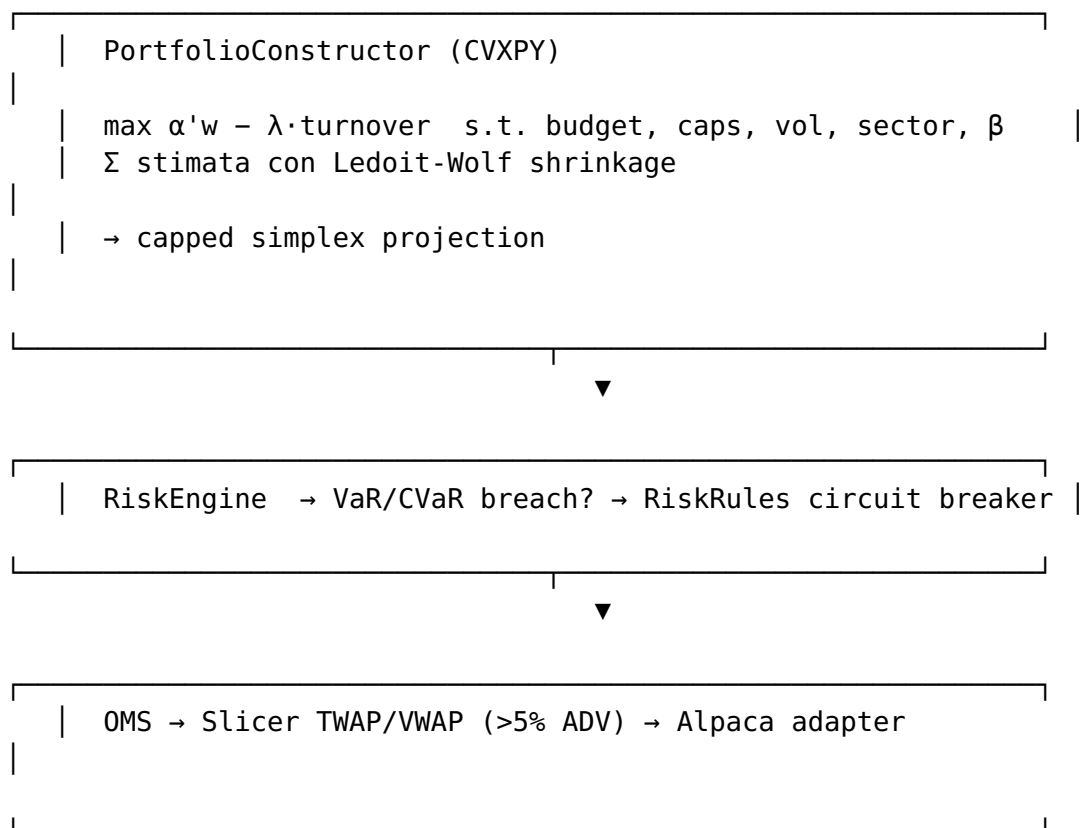
Metrica	Formula	File:linea
Sharpe annualizzato	$252 \cdot \bar{r}_{\text{exc}} / \sigma_r$	report.py: 129-146
Max Drawdown	$\min_t (E_t / \max_{s \leq t} E_s - 1)$	report.py: 148-160
Calmar	$\text{CAGR} / \text{MDD} $	report.py: 162-174
CAGR	$(E_T / E_0)^{1/T_y} - 1$	report.py: 622-629
IC (Spearman)	$\rho_S(\text{signal}_t, r_{t+1})$ cross-section	report.py: 176-217
IC Information Ratio	$\overline{\text{IC}} / \sigma_{\text{IC}} \cdot 252$	report.py: 279-358

Parte 3 — Diagrammi

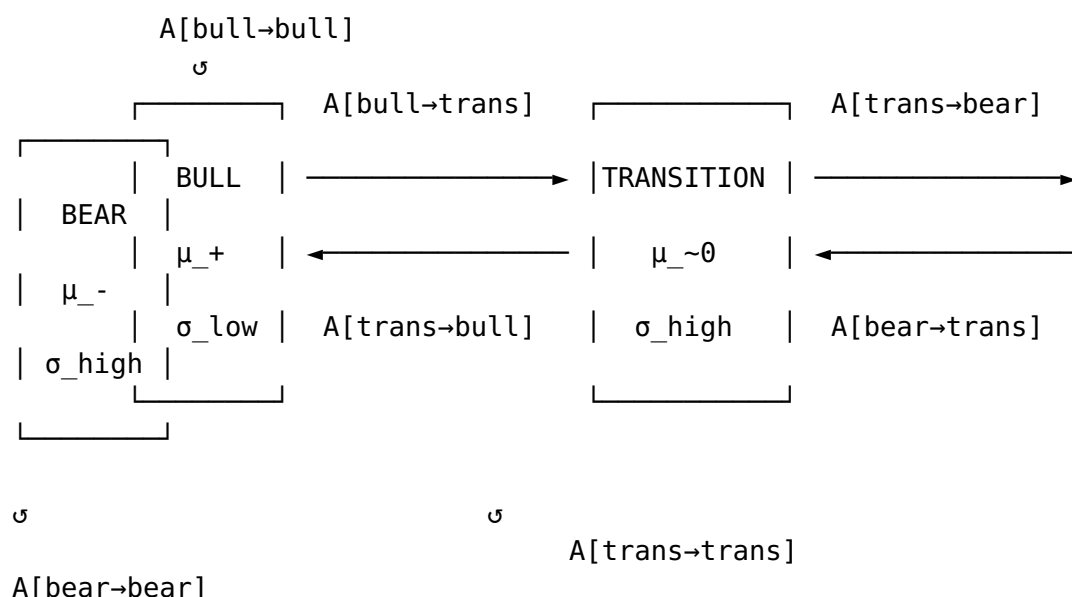
3.1 Diagramma di flusso end-to-end





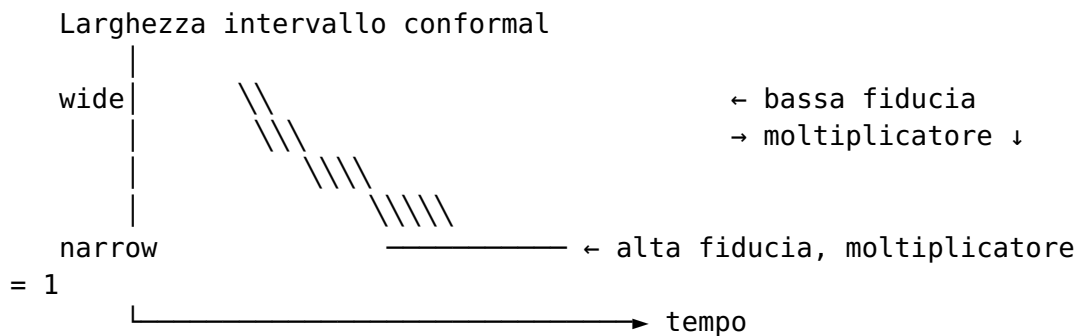


3.2 Stati HMM



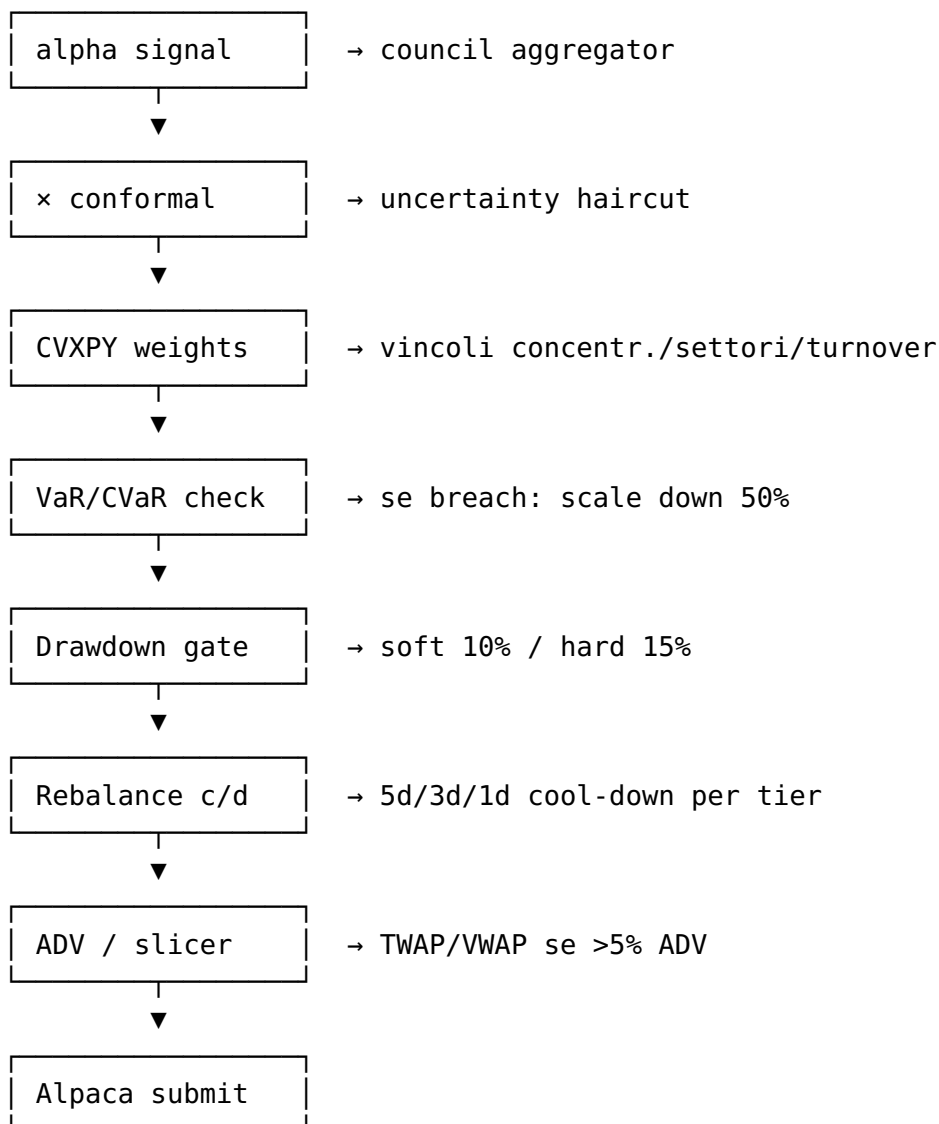
Emission: $P(x_t | z_t=k) = N(x_t | \mu_k, \Sigma_k)$
 Inference: Forward-backward (filtering+smoothing)
 Training: Baum-Welch EM, weekly Sunday 23:00 ET

3.3 Diagramma di robustezza Conformal sizing



`final_weight = w_optimizer × multiplier(width)`

3.4 Risk waterfall



2.8 Monitoring & alerting (council/monitor.py)

Check	Soglia	File:linea
Alpha decay	IC < 0.01 per ≥ 5 giorni consecutivi (CRITICAL se ≥ 10)	monitor.py:103-194
Feature drift	KS test $p < 0.05$ su $\geq 20\%$ delle top-10 feature, baseline 60d	monitor.py:200-330
SHAP stability	Jaccard overlap top-10 < 0.70 vs baseline	monitor.py:336-446
Regime change	regime cambia $\wedge$ transition_prob ≥ 0.70	monitor.py:452-527
Escalation	≥ 3 alert simultanei → tutti CRITICAL	monitor.py:672-679

Dispatch: INFO solo log, WARNING su dashboard, CRITICAL via email Gmail SMTP con deadletter queue (alerts.py:98-295).

2.9 Tabella iperparametri “nascosti” (non in CLAUDE.md)

Aspetto	Valore	File:linea
LGBM early stopping patience	50 iter	technical.py:261
Validation split LGBM	15% tail	technical.py:244
Min samples per fold CPCV	30 train, 5 test	technical.py:235
SHAP sample size	min(500, 15%)	technical.py:298
Ridge α nel conformal	1.0	conformal.py:88
Conformal CV folds	5	conformal.py:91
EWM halflife IC Sharpe	20 giorni	aggregator.py:514
Soft Sharpe floor aggregator	0.1	aggregator.py:437
Sentiment γ search bounds	[0.30, 0.95]	sentiment.py:310
FinBERT batch size	32	models.yaml:29
Min rebalance cool-down	1-5 giorni size-dep.	portfolio.py:298-299

Aspetto	Valore	File:linea
Signal strength filter	$ z z \geq 0.20$	portfolio.py:310
Sector cap	35% (dinamico)	portfolio.py:81
Conformal coverage	0.85	conformal.py:47
Position multiplier range	[0.2, 2.0]	conformal.py:184

Parte 4 — Analisi critica AS-IS (punti di debolezza)

1. **Modelli statici / re-train sporadico:** LGBM e HMM hanno checkpoint scaricati da disco; nessun online learning. In regimi nuovi (es. covid-style shock) il modello impiega settimane prima di adattarsi.
2. **Sentiment limitato a FinBERT su RSS:** FinBERT è 2019, vocabolario limitato, non cattura tono di earnings call, regulatory filings, X/Reddit.
3. **Council a 3 modelli con pesi lineari:** l'aggregator usa pesatura lineare. Non cattura interazioni non lineari (es. "sentiment è informativo solo quando regime in transizione").
4. **Covarianza Ledoit-Wolf su 90 giorni:** troppo corta per fat tails e troppo lunga per shock. Manca conditional covariance (DCC-GARCH).
5. **Vincoli statici:** max_position, sector_cap, turnover sono parametri scalari, non funzionali della volatilità realizzata.
6. **Conformal Jackknife+** è prediction-set; non è un quantile condizionato (CQR). Calibra coverage marginale ma non locale.
7. **Almgren-Chriss "tabulare":** il modello di slippage è una lookup table per ticker, non un modello calibrato sui propri fills.
8. **Nessun execution feedback loop:** la realized slippage non torna a calibrare il modello di costo pre-trade.
9. **Backtest senza walk-forward MLops:** una sola run su start..end. Manca purged + embargoed walk-forward con re-fit periodico.
10. **No causal inference:** il sistema correla feature → return, ma non separa effetti causali. Vulnerabile a spurious patterns.
11. **Single-asset modelling:** tutti i modelli sono per-ticker; non c'è un cross-asset attention/graph model.
12. **Mancanza di stress testing scenario-based:** VaR è statistico; non ci sono historical replay di scenari (LTCM, '08, '20).

Parte 5 — Proposte disruptive TO-BE

5.1 Modelli alpha di nuova generazione

5.1.1 Sostituire LightGBM con Temporal Fusion

Transformer (TFT) o **PatchTST** - Cattura dipendenze cross-asset e cross-time - Output multi-horizon nativo + interpretabilità via variable selection network - Matematica: $z_t = \text{LSTM}(x_{1:t}) + \text{Self-Attn}(x_{1:t})$ con gating $\sigma(W_g x) \sigma(W_o x)$

5.1.2 Sentiment: FinBERT → FinGPT/FinMA + RAG su filings

- Encoder LLM domain-tuned (FinMA-7B) + retrieval da 10-K/10-Q
- Capacità “zero-shot” su nuovi eventi (M&A, guidance) -
Aggiungere **multimodal**: parsing trascrizioni earnings call (audio→testo via Whisper + sentiment via LLM)

5.1.3 Regime: HMM → Switching State-Space Model con

Variational Inference - Sostituire 3 stati discreti con regime continuo $z_t \in \mathbb{R}^d$ via Deep State Space (DSS / S4) - $z_t = A\theta(z_{t-1}) + \epsilon_t$, $\epsilon_t \sim \mathcal{N}(0, \Sigma)$ con $A\theta A^T \Sigma^{-1}$ NN - Inference variazionale: amortized ELBO

5.1.4 Aggiungere quarto modello: Microstructure / Order

Flow Imbalance - OFI = (delta volume bid) - (delta volume ask) su L2 book - Highly predictive a horizon intraday - Già parzialmente disponibile via intraday/market_data.py

5.2 Council aggregation non lineare

5.2.1 Mixture-of-Experts (MoE) gating regime-aware -

Sostituire pesatura lineare con gating network
 $g_\theta(x_t, \text{regime}_t) = \frac{\exp(\theta_k(x_t, \text{regime}_t))}{\sum_k \exp(\theta_k(x_t, \text{regime}_t))}$
 $\hat{y} = \sum_k g_k(x) \cdot f_k(x)$ - Gating addestrato su backtest IC reward (REINFORCE / PPO)

5.2.2 Stacking con meta-learner conformal - Meta-learner

XGB/Linear su predizioni dei 3+ modelli base - Conformal calibration end-to-end via **Conditional Quantile Regression (CQR)** invece di Jackknife+: $\hat{C}_\alpha(x) = [q_{1-\alpha/2}(x), q_{1+\alpha/2}(x)]$
 $\hat{C}_\alpha(x) = [q_{1-\alpha/2}(x), q_{1+\alpha/2}(x)]$ - Garanzia coverage **condizionata** (più stretto in zone “facili”, più largo in code).

5.3 Portfolio construction evoluto

5.3.1 Risk-parity adattivo + Hierarchical Risk Parity (HRP, López de Prado) - Clustering gerarchico su matrice di correlazione → quasi-diagonalizzazione → recursive bisection - Robusto al condition number alto (non serve invertire Σ) - Iniettare HRP come **prior** del CVXPY (vincolo soft)

5.3.2 Robust optimization (Ben-Tal, Nemirovski) - Sostituire $\max_{\alpha} \tau_w \backslash \max \alpha^{\top} w$ con $\max_{\alpha \in U} \alpha^{\top} w \backslash \max_{\alpha \in U} \alpha^{\top} w$ su uncertainty set $U = \{\alpha: \|\alpha - \hat{\alpha}\|_{\Sigma^{-1}} \leq \kappa\}$ $U = \{\alpha: \|\alpha - \hat{\alpha}\|_{\Sigma^{-1}} \leq \kappa\}$ → equivalente a: $\max_w \alpha^{\top} w - \kappa w^{\top} \Sigma w \backslash \max_w \hat{\alpha}^{\top} w - \kappa \sqrt{w^{\top} \Sigma w}$ - Naturalmente penalizza pesi su asset con stima α incerta

5.3.3 Differentiable portfolio (cvxpylayers) - Rendere l'optimizer **differenziabile** end-to-end con il modello α - Loss = realized Sharpe → backprop dentro il QP - Permette di addestrare α + portfolio insieme (decision-focused learning)

5.3.4 Conditional covariance via DCC-GARCH o GNN - DCC: $\Sigma_t = D_t R_t D_t$ $\Sigma_t = D_t R_t D_t$ con D_t univariate GARCH e R_t dinamica - Alternativa: **Graph Neural Network** con edge = correlation, node feat = volatility regime

5.4 Esecuzione di nuova generazione

5.4.1 RL-based execution (Almgren-Chriss → Deep Hedging / RL) - Agente PPO su simulator di order book (LOB) per slicing ottimo - Reward = - implementation shortfall - Feature: book depth, recent prints, volatility, time-of-day

5.4.2 Self-calibrating cost model - Loop: realized slippage da OMS → fit online di κ in $\sigma_Q / \sqrt{Q/V}$ κ via Kalman filter - Aggiorna `transaction_costs.py` table giornalmente - Garantisce che backtest cost $\approx$ live cost (riduce overfit a costi pessimistici)

5.4.3 Smart Order Routing multi-venue - Oggi solo Alpaca. Aggiungere IBKR + Coinbase per crypto + Tradeweb su fixed income - Routing decisione: minimizza `expected_cost` + $\lambda \cdot$ urgency

5.5 Risk management

5.5.1 Stress test con generative scenarios - Generative model (VAE / Diffusion) su returns storici, condizionato su regimi rari - Genera 10k scenari per VaR/CVaR robust - Sostituisce VaR storico (sample-limited) con VaR generativo

5.5.2 Causal inference per drift detection - Algoritmi tipo **PCMCI / causal discovery** su feature-returns - Allerta quando struttura causale cambia (più informativo del KS test su distribuzioni marginali)

5.5.3 Crash early warning via topological data analysis - Persistent homology su returns multivariati - Spike in β_1 (loops) → segnale di market stress imminente (cf. Gidea-Katz 2018)

5.6 Infrastruttura & MLOps

5.6.1 Walk-forward CI con purged-embargoed CV - GitHub Action settimanale: re-fit modelli + run backtest WF + open PR se IR migliora - Champion-challenger automatico

5.6.2 Online learning con River / TensorFlow Decision Forests streaming - Sostituire `lgbm_latest.pkl` con update incrementale daily - Concept drift detection (ADWIN / DDM) → trigger retrain pesante

5.6.3 Feature store evoluto: bi-temporal → tri-temporal (Tecton/Feast pattern) - Aggiungere `arrival_time` separato da `transaction_time` → distingue “quando il dato è stato pubblicato” vs “quando l’abbiamo ingestito” - Permette analisi as-of “se fossimo stati live” più rigorosa

5.6.4 Observability: OpenTelemetry + Grafana - Distributed tracing dei segnali (ingest → orders) - Latency SLO per layer - Audit trail completo per compliance

5.7 Roadmap proposta (priorità)

Priorità	Tema	Stima impatto	Stima sforzo
● Alta	Self-calibrating cost model + RL execution	+20-50 bps Sharpe netto	1-2 mesi
● Alta	Conformal CQR + decision-focused learning	+0.2 Sharpe	2 mesi
● Media	DCC-GARCH covariance + robust opt.	drawdown -20%	3-6 sett.
● Media		IC +0.02-0.05	2-3 mesi

Priorità	Tema	Stima impatto	Stima sforzo
	TFT / PatchTST + MoE gating		
● Bassa	Microstructure model + smart routing	execution +5-10 bps	3-6 mesi
● Bassa	Generative stress testing + causal DD	risk insight	2 mesi

Verifica & utilizzo del report

- Il file è auto-contenuto: nessuna esecuzione di codice richiesta.
- Per validare i riferimenti, l'utente può aprire i file menzionati in VSCode con i jump-to-line (file:linea).
- Le formule LaTeX si renderizzano nei rendering Markdown standard (GitHub, MkDocs, Notion).
- I diagrammi ASCII si visualizzano correttamente in monospace.

Esecuzione: export in PDF (richiesto dall'utente)

Passi che verranno eseguiti dopo l'approvazione del piano:

1. **Copia il report markdown nel repository** in docs/codebase_analysis.md (cartella docs/ da creare se assente)
2. **Genera PDF** dal markdown. Strategia in ordine di preferenza, con fallback automatico:
 - `pandoc docs/codebase_analysis.md -o docs/codebase_analysis.pdf --pdf-engine=xelatex --toc -V geometry:margin=2cm`
 - Fallback se LaTeX non disponibile: `pandoc ... --pdf-engine=wkhtmltopdf`
 - Fallback ulteriore: `pandoc ... -t html poi weasyprint / chromium --headless --print-to-pdf`
 - Ultimo fallback: installazione `pip install markdown-pdf` e uso di markdown-pdf CLI
3. **Invia il PDF all'utente** via SendUserFile (status: normal)

4. **Commit & push** del markdown + PDF (entrambi) sul branch `claude/codebase-algorithm-analysis-JmoBp` come richiesto da `CLAUDE.md`.
- Commit message: “docs: add full codebase algorithm analysis (AS-IS + TO-BE) + PDF export”

Note operative: - Verifico prima quale strumento PDF è installato (`which pandoc, which wkhtmltopdf, pip list | grep -i pdf`) - Se nessuno è presente, installo `pandoc + texlive-xetex` via `apt-get` (richiede `sudo`, fallback `pip install pypandoc o markdown-pdf`) - Mantengo i diagrammi ASCII così come sono nel PDF (font monospace via CSS/LaTeX). Opzionalmente, posso convertirli in immagini Mermaid se l’utente lo richiede in seguito

Prossimi passi consigliati

1. Discutere quali aree TO-BE prioritizzare (sezione 5.7).
2. Per ogni iniziativa selezionata, aprire un design doc dedicato.
3. Convertire il diagramma di flusso ASCII in Mermaid per la documentazione del repo (`docs/architecture.md`).